@go_evangelist

Go Testing Handbook

A complete guide to **testing in Go**

Go 1.24+ | Practical | Production-ready

Why?

> “Writing tests is easy. Writing maintainable tests is much harder.”

Easy to write

```
func TestAdd(t *testing.T) {
    got := Add(2, 3)

    if got != 5 {
        t.Fatalf("expected 5,
          got %d", got)
    }
}
```

A few lines of code.
Instant feedback.

VS

Hard to maintain

```
tests/
  user_test.go
  user_edge_cases_test.go
  user_validation_test.go
  user_repository_test.go
  user_service_test.go
  ...
  and more...
```

As tests grow, complexity grows.
Duplication. Hard to read.

This handbook will help you write tests that are **clean**, **maintainable**, and **built to last**.

@go_evangelist

What you'll learn

Practical. Hands-on. Production-ready.

Table-Driven Testing

Write more tests with less code using data-driven test cases.

```
tests := []struct {
   name string
   input int
   want  int
}{...}
```

Subtests

Organize tests with t.Run for clearer output and better test reporting.

```
t.Run("case 1", func(t *testing.T) {
   // test logic
})
t.Run("case 2", func(t *testing.T) {
   // test logic
})
```

Test Helpers

Reduce repetition by using helper functions to keep tests clean and DRY.

```
func assertEqual(t *testing.T, got, want int) {
  t.Helper()
  if got != want {
     t.Fatalf("got %d, want %d", got, want)
  }
}
```

Coverage

Measure and improve your test coverage with built-in tools.

```
$ go test -cover ./...

coverage: 87.3% of statements
```

Benchmarks

Measure performance and prevent regressions using Go benchmarks.

```
func BenchmarkAdd(b *testing.B) {
  for i := 0; i < b.N; i++ {
     Add(1, 2)
  }
}
```

Parallel Tests

Speed up your test suite by running tests in parallel safely.

```
func TestSomething(t *testing.T) {
  t.Parallel()
  // test logic
}
```

Fuzz Testing

Automatically find edge cases and bugs with Go's fuzz testing.

```
func FuzzParse(f *testing.F) {
  f.Add("hello")
  f.Fuzz(func(t *testing.T, s string) {
     Parse(s)
  })
}
```

Mocking

Isolate external dependencies and test your code in complete isolation.

```
// Use mocks to simulate
// external behavior
mockService := NewMockService()
```

Best Practises

Follow proven testing practices to write reliable, maintainable tests.

- Keep tests fast and reliable
- Use meaningful test names
- Test behavior, not implementation
- Keep it simple and readable

From fundamentals to advanced techniques.
Everything you need to test Go code like a pro.

@go_evangelist

Who is it for?

This handbook is for anyone who wants to write **better, cleaner, and more maintainable tests in Go.**

Go Developers

Write better tests, avoid common mistakes, and follow **idiomatic Go** testing practices.

Backend Engineers

Building REST or gRPC services? Learn how to test your code with **confidence**.

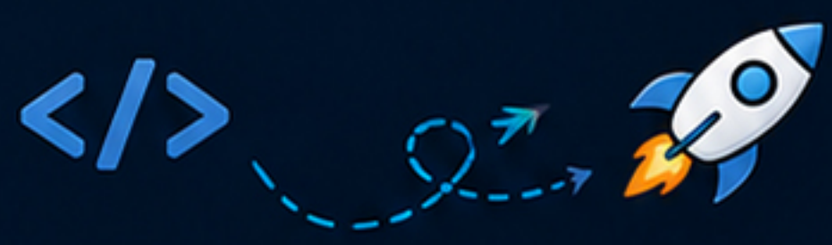
Students

Strengthen your skills and build a **solid foundation** in Go testing.

Interview Preparation

Master the testing topics that are frequently asked in Go **technical interviews.**

No matter your background, this handbook will help you become a **better Go engineer.**



5/6

@go_evangelist

Available now

INTRODUCTION

Available now!

Get an overview of the handbook, its structure, goals, and how to make the most of it.

- Purpose of this handbook
- Who it's for
- Recommended learning path

CHAPTER 1: TABLE-DRIVEN TESTING

Coming next!

Learn one of the most important and idiomatic testing patterns in Go.

- Write more tests with less code
- Improve readability and maintainability
- Cover more scenarios, confidently

```go
tests := []struct {
  name  string
  input  int
  want   int
} {
  {name: "case 1", input: 2, want: 4},
  {name: "case 2", input: 3, want: 6},
  {name: "case 3", input: 10, want: 20},
}

for _, tt := range tests {
  t.Run(tt.name, func(t *testing.T) {
    got := Add(tt.input)
    if got != tt.want {
      t.Errorf("got %d, want %d", got, tt.want)
    }
  })
}
```

New chapters will be released regularly.
Follow along and level up your Go testing skills!

Let's build better tests.

📖 Read it on GitHub

Deep dive into the full guide with examples, code and **best practices**.

🔗 Link in bio

Find the repository and start **exploring**.

💬 Follow for the next chapters

More Go tips, snippets & practical tricks **coming your way**.

Keep learning.
Keep shipping. Keep **building with Go**.

Enjoyed this?
Like, comment & share to support the series!

Like Comment Share Save